

Trường Đại học Công nghệ Thông tin – ĐHQG-HCM

Khoa Mạng máy tính & Truyền thông

Thực hành Giải thuật xử lý song song và phân bổ

Báo cáo thực hành giải thuật xử lý song song và phân bổ

Mã nhóm: MÃ NHÓM

Key: KEY

GVHD: Lê Minh Khánh Hội

Lê Phạm Hoàng Trung

Ngày nộp: Ngày 25 tháng 5 năm 2026

Thông tin thành viên

STT	Thành viên	Phân công chính
1	Họ tên – MSSV – nhiệm vụ chính	[CẦN ĐIỀN: challenge/phần việc]
2	Họ tên – MSSV – nhiệm vụ chính	[CẦN ĐIỀN: challenge/phần việc]
3	Họ tên – MSSV – nhiệm vụ chính	[CẦN ĐIỀN: challenge/phần việc]
4	Họ tên – MSSV – nhiệm vụ chính	[CẦN ĐIỀN: challenge/phần việc]

TP. Hồ Chí Minh, Ngày 25 tháng 5 năm 2026

Cam kết

Nhóm cam kết mã nguồn nộp lên hệ thống là phiên bản cuối cùng của nhóm, tuân thủ danh sách thư viện được phép dùng trong đề bài và không nộp mã nguồn tuân tự lên hệ thống chấm. Các số liệu hiệu năng trong báo cáo được đo từ môi trường thực nghiệm được mô tả tại Mục 1.

[CẦN ĐIỀN: Điền thêm cam kết hoặc chữ ký nếu giảng viên yêu cầu.]

Mục lục

1	Tổng quan yêu cầu và phương pháp đánh giá	6
1.1	Mục tiêu báo cáo	6
1.2	Phạm vi nội dung	6
1.3	Môi trường thực nghiệm	6
1.4	Phương pháp đo hiệu năng	7
1.5	Tóm tắt mã nguồn hiện có	8
2	Challenge 1: Tính tổng mảng số nguyên	8
2.1	Hiện thực đã xây dựng	8
2.2	Phân tích song song hóa	9
2.3	Thiết kế thuật toán song song	9
2.4	Lưu đồ thuật toán	10
2.5	Giải thích vai trò các bước	10
2.6	Tiềm năng tăng tốc và ví dụ minh họa	10
2.7	Dữ liệu, phụ thuộc và chi phí	11
2.8	Đánh giá hiệu năng	11
3	Challenge 2: Số Fibonacci modulo	11
3.1	Hiện thực đã xây dựng	12
3.2	Phân tích song song hóa	12
3.3	Thiết kế thuật toán song song	12
3.4	Lưu đồ thuật toán	13
3.5	Giải thích vai trò các bước	13
3.6	Tiềm năng tăng tốc và ví dụ minh họa	14
3.7	Dữ liệu, phụ thuộc và chi phí	14
3.8	Đánh giá hiệu năng	14
4	Challenge 3: Nhân hai ma trận	14
4.1	Hiện thực đã xây dựng	15
4.2	Phân tích song song hóa	15
4.3	Thiết kế thuật toán song song	15
4.4	Lưu đồ thuật toán	16
4.5	Giải thích vai trò các bước	16
4.6	Tiềm năng tăng tốc và ví dụ minh họa	17
4.7	Dữ liệu, phụ thuộc và chi phí	17
4.8	Đánh giá hiệu năng	18

5	Challenge 4: Sắp xếp mảng một chiều	18
5.1	Hiện thực đã xây dựng	18
5.2	Phân tích song song hóa	18
5.3	Thiết kế thuật toán song song	19
5.4	Lưu đồ thuật toán	20
5.5	Giải thích vai trò các bước	20
5.6	Tiềm năng tăng tốc và ví dụ minh họa	21
5.7	Tính ổn định	21
5.8	Dữ liệu, phụ thuộc và chi phí	21
5.9	Đánh giá hiệu năng	21
6	Challenge 5: Tìm kiếm trong mảng số nguyên lớn	22
6.1	Hiện thực đã xây dựng	22
6.2	Phân tích song song hóa	22
6.3	Thiết kế thuật toán song song	23
6.4	Lưu đồ thuật toán	24
6.5	Giải thích vai trò các bước	24
6.6	Tiềm năng tăng tốc và ví dụ minh họa	25
6.7	Dữ liệu, phụ thuộc và chi phí	25
6.8	Đánh giá hiệu năng	25
7	Challenge 6: Phân tích không gian nhúng	26
7.1	Hiện thực đã xây dựng	26
7.2	Phân tích song song hóa	26
7.3	Thiết kế thuật toán song song	26
7.4	Lưu đồ thuật toán	27
7.5	Giải thích vai trò các bước	28
7.6	Tiềm năng tăng tốc và ví dụ minh họa	28
7.7	Dữ liệu, phụ thuộc và chi phí	28
7.8	Đánh giá hiệu năng	29
8	Kết luận	29
8.1	Tổng hợp kết quả	29
8.2	Nhận định chung	30
8.3	Checklist trước khi nộp	30
A	Phụ lục: Mẫu đo hiệu năng	30
B	Phụ lục: Mẫu phiên bản tuần tự tham chiếu	31

C Phụ lục: Checklist nội dung theo đề

32

Danh sách bảng

1	Thông tin môi trường đo hiệu năng	7
2	Tóm tắt hiện trạng mã nguồn theo từng challenge	8
3	Kết quả đo hiệu năng cho Challenge 1	11
4	Kết quả đo hiệu năng cho Challenge 2	14
5	Kết quả đo hiệu năng cho Challenge 3	18
6	Kết quả đo hiệu năng cho Challenge 4	21
7	Kết quả đo hiệu năng cho Challenge 5	25
8	Kết quả đo hiệu năng cho Challenge 6	29
9	Tổng hợp speedup tốt nhất sau khi đo	29
10	Phiên bản tuần tự tham chiếu đề xuất	31
11	Đối chiếu yêu cầu đề bài với báo cáo	32

Danh sách hình vẽ

1	Luồng xử lý song song của Challenge 1	10
2	Luồng xử lý song song của Challenge 2	13
3	Luồng xử lý song song của Challenge 3	16
4	Luồng xử lý song song của Challenge 4	20
5	Luồng xử lý song song của Challenge 5	24
6	Luồng xử lý song song của Challenge 6	27

1 Tổng quan yêu cầu và phương pháp đánh giá

1.1 Mục tiêu báo cáo

Bài thực hành yêu cầu xây dựng các thuật toán xử lý song song cho sáu bài toán có kích thước dữ liệu lớn. Trọng tâm của báo cáo không chỉ là mô tả kết quả hiện thực, mà còn làm rõ cơ sở thiết kế: phần việc nào có thể thực thi đồng thời, dữ liệu được phân chia như thế nào, các tiến trình phối hợp ra sao và những yếu tố nào ảnh hưởng đến hiệu năng.

Các chương trình được hiện thực bằng Python theo giao diện chung là hàm **MAIN**. Mỗi hàm nhận đường dẫn tệp đầu vào, xử lý theo yêu cầu của từng challenge và trả về kết quả đúng định dạng để hệ thống chấm có thể kiểm tra tự động.

1.2 Phạm vi nội dung

Theo yêu cầu của đề bài, mỗi challenge trong báo cáo được trình bày theo cùng một cấu trúc:

- **Mô tả bài toán:** tóm tắt dữ liệu vào, dữ liệu ra và điều kiện cần thỏa mãn.
- **Phân tích song song hóa:** xác định các đơn vị công việc độc lập, phương án chia việc, cách phân phối và truy cập dữ liệu.
- **Thiết kế thuật toán:** trình bày luồng xử lý bằng mô tả, pseudocode và lưu đồ minh họa; đồng thời giải thích vai trò của từng bước và cách các tiến trình phối hợp.
- **Đánh giá hiệu năng:** so sánh thời gian thực thi giữa phiên bản tuần tự tham chiếu và phiên bản song song, từ đó phân tích speedup và efficiency.
- **Ví dụ minh họa:** đưa ví dụ nhỏ để làm rõ cách thuật toán xử lý dữ liệu và tạo kết quả.

1.3 Môi trường thực nghiệm

Các số liệu trong báo cáo cần được đo trên cùng một môi trường để bảo đảm tính nhất quán. Bảng 1 là nơi ghi lại cấu hình thực nghiệm đã sử dụng.

Bảng 1: Thông tin môi trường đo hiệu năng

Hạng mục	Giá trị
CPU	[CẦN ĐIỀN: Tên CPU, số core/số thread]
RAM	[CẦN ĐIỀN: Dung lượng RAM]
Hệ điều hành	[CẦN ĐIỀN: OS và phiên bản]
Python	[CẦN ĐIỀN: Phiên bản Python trên máy đo hoặc server]
Số worker mặc định	[CẦN ĐIỀN: Giá trị multiprocessing.cpu_count() hoặc cấu hình cố định]
Lệnh/công cụ đo thời gian	[CẦN ĐIỀN: time.perf_counter hoặc công cụ đo tương đương]

1.4 Phương pháp đo hiệu năng

Đối với mỗi kích thước dữ liệu, chương trình được chạy nhiều lần trên cùng một cấu hình. Thời gian thực thi trung bình được tính theo công thức:

$$T = \frac{1}{k} \sum_{i=1}^k T_i,$$

trong đó k là số lần lặp. Khi cần tránh nhiễu do khởi tạo cache hoặc nạp module, lần chạy đầu tiên có thể được dùng làm warm-up và không đưa vào kết quả trung bình.

Hai chỉ số chính được dùng để đánh giá là:

$$S_p = T_{\text{seq}}/T_{\text{par}}, \quad E_p = S_p/p.$$

Trong đó T_{seq} là thời gian của phiên bản tuần tự tham chiếu, T_{par} là thời gian của phiên bản song song và p là số worker được sử dụng. Speedup phản ánh mức cải thiện thời gian, còn efficiency cho biết mức độ tận dụng tài nguyên xử lý.

1.5 Tóm tắt mã nguồn hiện có

Bảng 2: Tóm tắt hiện trạng mã nguồn theo từng challenge

Challenge	File	Đặc điểm hiện thực chính
1	challenge/challenge1.py	Chia dữ liệu đầu vào theo byte-range, mỗi worker lọc và cộng các giá trị hợp lệ, tiến trình chính gom tổng cục bộ.
2	challenge/challenge2.py	Xử lý độc lập từng truy vấn Fibonacci modulo; kết hợp fast doubling, bảng window và mảng chia sẻ RawArray.
3	challenge/challenge3.py	Chia phép nhân ma trận theo cụm dòng; sử dụng ma trận B đã chuyển vị để tăng tính liên tục khi truy cập dữ liệu.
4	challenge/challenge4.py	Tự hiện thực merge sort; sắp xếp song song các chunk ban đầu rồi merge kết quả theo thứ tự.
5	challenge/challenge5.py	Tìm khóa trực tiếp trên buffer byte; chia vùng dữ liệu thành các khoảng byte, worker trả về offset ứng viên và tiến trình chính quy đổi offset nhỏ nhất về chỉ số mảng.
6	challenge/challenge6.py	Xử lý độc lập từng bộ điểm 2D; dùng mã hóa điểm và hash grid để tìm khoảng cách gần nhất.

2 Challenge 1: Tính tổng mảng số nguyên

Yêu cầu bài toán

Cho dãy A gồm N giá trị thực. Chương trình cần tính tổng các phần tử đồng thời thỏa ba điều kiện: có giá trị toán học là số nguyên, lớn hơn 0 và chia hết cho 3. Kết quả trả về là một số nguyên, trong đó miền giá trị có thể vượt giới hạn 32-bit.

2.1 Hiện thực đã xây dựng

Trong challenge/challenge1.py, dữ liệu đầu vào được đọc ở dạng byte nhằm giảm chi phí chuyển đổi chuỗi không cần thiết. Sau khi xác định vị trí kết thúc dòng đầu tiên chứa N ,

phần dữ liệu còn lại được chia thành nhiều đoạn liên tiếp theo byte. Mỗi đoạn được giao cho một worker để xử lý độc lập thông qua hàm `_process_indices`.

Tại mỗi worker, các token trong đoạn được tách bằng khoảng trắng, chuyển sang kiểu số thực và kiểm tra lần lượt các điều kiện của đề bài. Worker chỉ trả về tổng cục bộ của đoạn mà nó phụ trách. Tiến trình chính thực hiện bước gom kết quả bằng cách cộng các tổng cục bộ này.

2.2 Phân tích song song hóa

Bài toán có tính độc lập dữ liệu rất cao. Việc kiểm tra một phần tử không phụ thuộc vào giá trị hoặc trạng thái của các phần tử khác. Do đó, mô hình song song phù hợp là *map-reduce*: giai đoạn map lọc và cộng cục bộ trên từng đoạn dữ liệu, còn giai đoạn reduce cộng các tổng cục bộ thành kết quả cuối cùng.

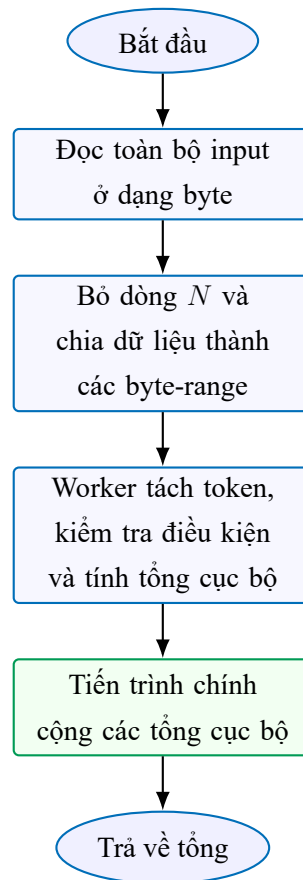
Cách chia theo byte-range giúp tránh phải parse trước toàn bộ dãy số. Tuy nhiên, để bảo đảm một token không bị cắt đôi giữa hai worker, điểm kết thúc của mỗi đoạn được dịch đến khoảng trắng gần nhất. Thiết kế này giảm chi phí tiền xử lý, đồng thời vẫn giữ được tính đúng đắn của quá trình tách token.

2.3 Thiết kế thuật toán song song

Listing 1: Pseudocode song song cho Challenge 1

```
1 MAIN(path):
2     raw = read_all_bytes(path)
3     data_start = locate_first_data_byte(raw)
4     ranges = split_byte_ranges_at_token_boundary(raw, data_start)
5
6     partial_sums = pool.starmap(process_range, ranges)
7     return sum(partial_sums)
8
9 process_range(start, end):
10     local_sum = 0
11     for token in raw[start:end].split():
12         value = float(token)
13         if value > 0 and value.is_integer() and int(value) % 3 == 0:
14             local_sum += int(value)
15     return local_sum
```

2.4 Lưu đồ thuật toán



Hình 1: Luồng xử lý song song của Challenge 1

2.5 Giải thích vai trò các bước

Luồng xử lý của Challenge 1 gồm hai vai trò rõ ràng. Tiến trình chính chịu trách nhiệm đọc input, bỏ qua dòng đầu chứa N , xác định các biên token và tạo danh sách byte-range. Việc chuẩn bị này đảm bảo mỗi worker nhận một đoạn dữ liệu hợp lệ, không bắt đầu hoặc kết thúc ở giữa một số.

Các worker thực hiện cùng một thủ tục trên các đoạn khác nhau: tách token, chuyển token sang số thực, kiểm tra ba điều kiện của đề bài và cộng vào biến tổng cục bộ. Worker không cần biết kết quả của worker khác, vì một phần tử hợp lệ chỉ đóng góp đúng một lần vào đúng đoạn chứa nó. Sau khi tất cả worker trả về, tiến trình chính cộng các tổng cục bộ. Bước reduce này là phép cộng kết hợp, nên thứ tự gom kết quả không làm thay đổi giá trị cuối cùng.

2.6 Tiềm năng tăng tốc và ví dụ minh họa

So với phiên bản tuần tự phải quét toàn bộ token trên một tiến trình, phiên bản song song có thể rút ngắn thời gian khi N lớn vì phân kiểm tra từng token độc lập hoàn toàn. Speedup kỳ

vọng tăng khi số lượng token đủ lớn để bù chi phí tạo process và truyền tham số. Với input nhỏ, overhead của multiprocessing có thể làm chương trình song song chậm hơn.

Ví dụ, với dãy 3 4.5 -6 12.0 7 18, nếu chia thành hai đoạn, worker thứ nhất có thể xử lý 3 4.5 -6 và trả về 3, worker thứ hai xử lý 12.0 7 18 và trả về 30. Tiến trình chính cộng $3 + 30 = 33$, đúng với kết quả cần trả về. Ví dụ này cũng thể hiện vai trò của bước chia dữ liệu: mỗi token chỉ thuộc một worker, còn bước reduce chỉ gom các tổng cục bộ.

2.7 Dữ liệu, phụ thuộc và chi phí

- **Phân phối dữ liệu:** dữ liệu đầu vào được lưu trong biến toàn cục; worker chỉ nhận cặp chỉ số (start, end).
- **Phụ thuộc dữ liệu:** không có phụ thuộc giữa các phần tử; chỉ tồn tại bước reduce cuối cùng.
- **Cân bằng tải:** chia theo số byte giúp tải tương đối đều, nhưng có thể lệch nhẹ nếu độ dài token khác nhau nhiều.
- **Chi phí giao tiếp:** thấp, vì mỗi worker chỉ trả về một số nguyên đại diện cho tổng cục bộ.

2.8 Đánh giá hiệu năng

Bảng 3: Kết quả đo hiệu năng cho Challenge 1

Kích thước	Số worker	T_{seq} (s)	T_{par} (s)	Speedup	Efficiency
[CẦN ĐIỀN: size]	[CẦN ĐIỀN: p]	[CẦN ĐIỀN: tseq]	[CẦN ĐIỀN: tpar]	[CẦN ĐIỀN: speedup]	[CẦN ĐIỀN: eff]
[CẦN ĐIỀN: size]	[CẦN ĐIỀN: p]	[CẦN ĐIỀN: tseq]	[CẦN ĐIỀN: tpar]	[CẦN ĐIỀN: speedup]	[CẦN ĐIỀN: eff]
[CẦN ĐIỀN: size]	[CẦN ĐIỀN: p]	[CẦN ĐIỀN: tseq]	[CẦN ĐIỀN: tpar]	[CẦN ĐIỀN: speedup]	[CẦN ĐIỀN: eff]

Nhận xét. [CẦN ĐIỀN: So sánh thời gian tuần tự và song song. Cần làm rõ trường hợp input nhỏ có thể không đạt speedup do chi phí tạo tiến trình, trong khi input lớn thường khai thác tốt hơn tính độc lập của từng token.]

3 Challenge 2: Số Fibonacci modulo

Yêu cầu bài toán

Với N truy vấn A_i và một modulo cố định Q , chương trình cần trả về danh sách $F(A_i) \bmod Q$ theo đúng thứ tự xuất hiện trong input. Ràng buộc lớn của N và A_i khiến việc tính lặp tuyến tính từng số Fibonacci là không khả thi.

3.1 Hiện thực đã xây dựng

Trong `challenge/challenge2.py`, mỗi truy vấn Fibonacci được xử lý độc lập. Chương trình sử dụng hai kỹ thuật chính. Kỹ thuật thứ nhất là fast doubling, cho phép tính $F(n)$ với độ phức tạp $\mathcal{O}(\log n)$. Kỹ thuật thứ hai là bảng window, trong đó chỉ số n được tách thành phần thấp và phần cao để tái sử dụng các giá trị Fibonacci đã tiền xử lý.

Kết quả được ghi vào `multiprocessing.RawArray`. Cách tổ chức này giúp các worker ghi trực tiếp vào vùng kết quả đã được định vị bằng offset, từ đó bảo toàn thứ tự output mà không cần truyền một danh sách lớn về tiến trình chính.

3.2 Phân tích song song hóa

Các truy vấn $F(A_i) \bmod Q$ không phụ thuộc lẫn nhau, do đó đơn vị song song tự nhiên là từng đoạn truy vấn. Mỗi worker nhận một đoạn byte chứa nhiều truy vấn, tách token, tính kết quả và ghi vào vùng output tương ứng. Trước khi worker bắt đầu, các cấu trúc dùng chung như modulo và bảng window phải được khởi tạo hoàn chỉnh.

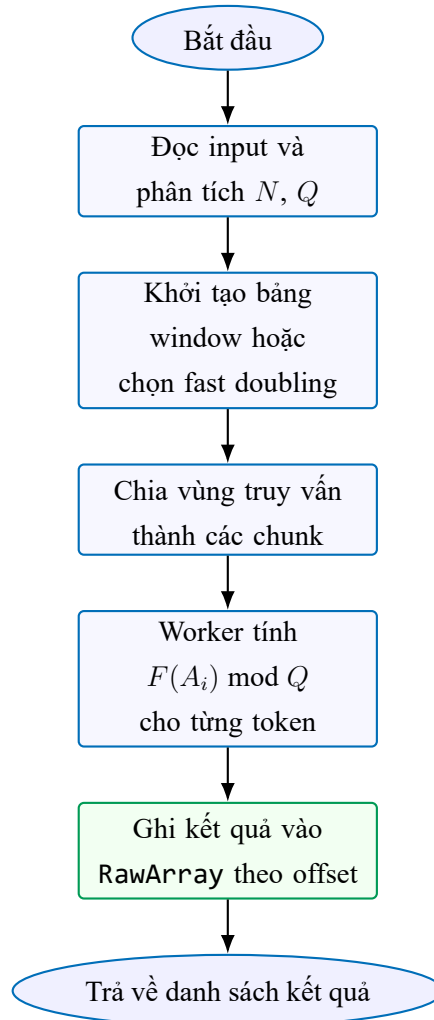
Việc chia theo byte-range giúp giảm chi phí tạo danh sách truy vấn trung gian. Để bảo toàn thứ tự, mỗi chunk đi kèm với `out_start`, là vị trí bắt đầu ghi kết quả trong `RawArray`.

3.3 Thiết kế thuật toán song song

Listing 2: Pseudocode song song cho Challenge 2

```
1 MAIN(path):
2     raw = read_all_bytes(path)
3     N, modulo, query_start = parse_header(raw)
4     prepare_fibonacci_tables_if_beneficial()
5
6     output = RawArray(size=N)
7     chunks = split_query_region(raw, query_start)
8
9     for each chunk in parallel:
10         tokens = raw[chunk.start:chunk.end].split()
11         for j, token in enumerate(tokens):
12             n = int(token)
13             output[chunk.out_start + j] = fib_mod(n, modulo)
14
15     return output[:]
```

3.4 Lưu đồ thuật toán



Hình 2: Luồng xử lý song song của Challenge 2

3.5 Giải thích vai trò các bước

Tiến trình chính thực hiện các bước có tính toàn cục: đọc N và modulo Q , lựa chọn cách tính Fibonacci, khởi tạo bảng window nếu số lượng truy vấn đủ lớn và cấp phát RawArray cho output. Các bước này phải hoàn tất trước khi tạo worker vì mọi worker đều phụ thuộc vào cùng giá trị modulo và cùng cấu trúc tra cứu.

Sau đó, vùng chứa các truy vấn được chia thành nhiều chunk. Mỗi chunk đi kèm vị trí bắt đầu trong output, nhờ đó worker có thể ghi kết quả của truy vấn thứ j trong chunk vào đúng ô $\text{out_start} + j$. Cách ghi này thể hiện tương tác giữa các tiến trình: worker không cần gửi lại toàn bộ danh sách kết quả, cũng không cần khóa đồng bộ, vì mỗi worker được cấp một vùng ghi không giao nhau.

3.6 Tiềm năng tăng tốc và ví dụ minh họa

Bài toán có tiềm năng tăng tốc tốt khi N lớn, vì các truy vấn Fibonacci độc lập. Nếu mỗi truy vấn được tính bằng fast doubling với chi phí $\mathcal{O}(\log A_i)$, tổng khối lượng tính toán đủ lớn có thể được chia cho nhiều worker. Bảng window còn giúp giảm chi phí lặp lại khi nhiều truy vấn có chỉ số lớn và cùng modulo.

Ví dụ, với $N = 6$ truy vấn và 3 worker, có thể chia thành ba chunk: worker 1 xử lý A_0, A_1 , worker 2 xử lý A_2, A_3 , worker 3 xử lý A_4, A_5 . Nếu input là 3, 4, 5, 6, 7, 8 với $Q = 10^9$, các worker lần lượt ghi các kết quả 2, 3, 5, 8, 13, 21 vào các vùng đã định sẵn. Khi tiến trình chính đọc lại RawArray, thứ tự output vẫn là thứ tự ban đầu của input.

3.7 Dữ liệu, phụ thuộc và chi phí

- **Phân phối dữ liệu:** input byte và bảng Fibonacci là dữ liệu đọc chung; output là mảng chia sẻ ghi theo vùng không giao nhau.
- **Phụ thuộc dữ liệu:** các truy vấn độc lập; phụ thuộc duy nhất là bước tiền xử lý bảng phải hoàn tất trước khi chạy worker.
- **Cân bằng tải:** nếu độ dài truy vấn tương đối đồng đều, chia theo byte-range tạo tải gần cân bằng.
- **Chi phí giao tiếp:** thấp vì worker ghi trực tiếp vào RawArray; chi phí chính nằm ở parse token, tính modulo và khởi tạo process.

3.8 Đánh giá hiệu năng

Bảng 4: Kết quả đo hiệu năng cho Challenge 2

Kích thước	Số worker	T_{seq} (s)	T_{par} (s)	Speedup	Efficiency
[CẦN ĐIỀN: size]	[CẦN ĐIỀN: p]	[CẦN ĐIỀN: tseq]	[CẦN ĐIỀN: tpar]	[CẦN ĐIỀN: speedup]	[CẦN ĐIỀN: eff]
[CẦN ĐIỀN: size]	[CẦN ĐIỀN: p]	[CẦN ĐIỀN: tseq]	[CẦN ĐIỀN: tpar]	[CẦN ĐIỀN: speedup]	[CẦN ĐIỀN: eff]
[CẦN ĐIỀN: size]	[CẦN ĐIỀN: p]	[CẦN ĐIỀN: tseq]	[CẦN ĐIỀN: tpar]	[CẦN ĐIỀN: speedup]	[CẦN ĐIỀN: eff]

Nhận xét. [CẦN ĐIỀN: Phân tích lợi ích của bảng window so với fast doubling thuần. Cần nêu rõ khi số truy vấn nhỏ, overhead dựng bảng và tạo tiến trình có thể lớn hơn lợi ích song song hóa.]

4 Challenge 3: Nhân hai ma trận

Yêu cầu bài toán

Cho hai ma trận vuông A và B kích thước $N \times N$, chương trình cần tính $C = A \times B$. Ngoài ma trận kết quả, chương trình phải trả về đường chéo chính, đường chéo phụ và tổng toàn bộ phần tử của C .

4.1 Hiện thực đã xây dựng

Trong `challenge/challenge3.py`, dữ liệu được đọc bằng generator `_int_stream` nhằm tránh tạo một danh sách token chuỗi quá lớn. Ma trận B được lưu dưới dạng chuyển vị B^T . Nhờ vậy, khi tính tích vô hướng giữa một dòng của A và một cột của B , chương trình thực chất truy cập hai danh sách theo thứ tự liên tiếp.

Đơn vị công việc được chọn là một cụm dòng của ma trận C . Hàm `_mul_chunk` nhận khoảng dòng, tính toàn bộ các phần tử thuộc khoảng đó, đồng thời trích xuất phần tương ứng của đường chéo chính, đường chéo phụ và tổng cục bộ.

4.2 Phân tích song song hóa

Sau khi A và B^T đã sẵn sàng, mỗi dòng C_i có thể được tính độc lập. Điều này cho phép chia ma trận kết quả theo các khoảng dòng không giao nhau. Tiến trình chính chỉ cần ghép các đoạn kết quả theo chỉ số dòng ban đầu.

So với chia theo từng ô (i, j) , chia theo dòng giảm số lượng task, giữ dữ liệu kết quả theo khối lớn hơn và thuận lợi cho việc trích xuất đường chéo. Tuy nhiên, vì mỗi worker trả về nhiều dòng của C , chi phí truyền dữ liệu và bộ nhớ là yếu tố cần được theo dõi khi N lớn.

4.3 Thiết kế thuật toán song song

Listing 3: Pseudocode song song cho Challenge 3

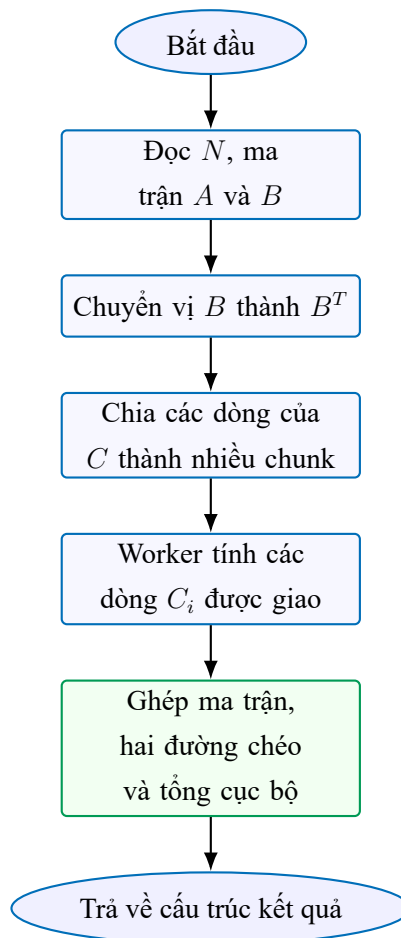
```
1 MAIN(path):
2     N = read_N(path)
3     A = read_matrix_A(path, N)
4     BT = read_transposed_matrix_B(path, N)
5     tasks = split_rows_into_chunks(N)
6
7     for each (start_row, end_row) in parallel:
8         rows = compute_C_rows(A, BT, start_row, end_row)
9         diag_main_part = extract_main_diagonal(rows)
10        diag_secondary_part = extract_secondary_diagonal(rows)
```

```

11     local_sum = sum(rows)
12     return start_row, rows, diag_parts, local_sum
13
14 merge rows and diagonal parts by start_row
15 return {matrix, diag_main, diag_secondary, total_sum}

```

4.4 Lưu đồ thuật toán



Hình 3: Luồng xử lý song song của Challenge 3

4.5 Giải thích vai trò các bước

Trong Challenge 3, bước chuyển vị B giữ vai trò quan trọng đối với hiệu năng. Khi B được lưu thành B^T , phép tính $C_{ij} = \sum_k A_{ik} B_{kj}$ trở thành tích vô hướng giữa dòng A_i và dòng B_j^T . Nhờ đó, worker truy cập hai dãy liên tiếp trong bộ nhớ thay vì phải nhảy theo cột của ma trận B ban đầu.

Phương pháp chia việc được chọn là chia theo cụm dòng của ma trận C . Mỗi worker nhận một khoảng dòng $[r_{\text{start}}, r_{\text{end}})$ và tính toàn bộ các phần tử C_{ij} trong khoảng đó. Trong lúc tính

từng dòng, worker đồng thời xác định phần tử đường chéo chính nếu $i = j$, phần tử đường chéo phụ nếu $i + j = N - 1$, và cộng các phần tử vào tổng cục bộ. Vì các khoảng dòng không giao nhau, không có hai worker nào cùng ghi hoặc cùng trả về một dòng của C .

Tiến trình chính nhận kết quả theo từng chunk, đặt các dòng vào đúng vị trí trong ma trận kết quả, ghép các phần đường chéo và cộng các tổng cục bộ. Như vậy, từng bước trong thuật toán đều đóng góp trực tiếp vào cấu trúc output: worker tạo các dòng của C , còn tiến trình chính bảo toàn thứ tự dòng và tổng hợp các giá trị phụ trợ.

4.6 Tiềm năng tăng tốc và ví dụ minh họa

Nhân ma trận có khối lượng tính toán lớn, xấp xỉ $\mathcal{O}(N^3)$. Vì vậy khi N tăng, phần tính toán trên mỗi worker tăng nhanh hơn chi phí tạo process và ghép kết quả, làm cho bài toán có tiềm năng speedup rõ rệt hơn các bài toán chỉ quét tuyến tính. Tuy nhiên, speedup không thể tăng tuyến tính mãi vì ma trận kết quả có kích thước $\mathcal{O}(N^2)$, gây áp lực bộ nhớ và chi phí truyền dữ liệu giữa process.

Ví dụ với $N = 2$, ma trận $A = \begin{bmatrix} 1 & 2 \\ 3 & 4 \end{bmatrix}$ và $B = \begin{bmatrix} 5 & 6 \\ 7 & 8 \end{bmatrix}$. Nếu chia mỗi worker một dòng,

worker 1 tính dòng 0 của C là $[19, 22]$, đồng thời ghi nhận phần tử đường chéo chính 19 và đường chéo phụ 22. Worker 2 tính dòng 1 là $[43, 50]$, ghi nhận đường chéo phụ 43 và đường chéo chính 50. Tiến trình chính ghép hai dòng thành C , thu được $\text{diag_main} = [19, 50]$, $\text{diag_secondary} = [22, 43]$ và tổng 134.

4.7 Dữ liệu, phụ thuộc và chi phí

- **Phân phối dữ liệu:** A và B^T được dùng như dữ liệu đọc chung; worker trả về các dòng C thuộc khoảng phụ trách.
- **Phụ thuộc dữ liệu:** các dòng của C độc lập, vì vậy không cần đồng bộ trong quá trình nhân.
- **Cân bằng tải:** chia theo dòng tạo tải tương đối đều do mỗi dòng có cùng số phép nhân-cộng.
- **Chi phí giao tiếp:** đáng kể khi N lớn, vì dữ liệu kết quả có kích thước $\mathcal{O}(N^2)$.
- **Giới hạn bộ nhớ:** cần theo dõi bộ nhớ khi thử các kích thước lớn như $N = 5000$.

4.8 Đánh giá hiệu năng

Bảng 5: Kết quả đo hiệu năng cho Challenge 3

N	Số worker	T_{seq} (s)	T_{par} (s)	Speedup	Efficiency
128	[CẦN ĐIỀN: p]	[CẦN ĐIỀN: tseq]	[CẦN ĐIỀN: tpar]	[CẦN ĐIỀN: speedup]	[CẦN ĐIỀN: eff]
256	[CẦN ĐIỀN: p]	[CẦN ĐIỀN: tseq]	[CẦN ĐIỀN: tpar]	[CẦN ĐIỀN: speedup]	[CẦN ĐIỀN: eff]
512	[CẦN ĐIỀN: p]	[CẦN ĐIỀN: tseq]	[CẦN ĐIỀN: tpar]	[CẦN ĐIỀN: speedup]	[CẦN ĐIỀN: eff]
1024	[CẦN ĐIỀN: p]	[CẦN ĐIỀN: tseq]	[CẦN ĐIỀN: tpar]	[CẦN ĐIỀN: speedup]	[CẦN ĐIỀN: eff]
5000	[CẦN ĐIỀN: p]	[CẦN ĐIỀN: tseq]	[CẦN ĐIỀN: tpar]	[CẦN ĐIỀN: speedup]	[CẦN ĐIỀN: eff]

Nhận xét. [CẦN ĐIỀN: Giải thích xu hướng khi N tăng. Cần làm rõ phần tính toán tăng nhanh giúp song song hóa có lợi hơn, nhưng chi phí bộ nhớ và truyền ma trận kết quả có thể làm speedup thấp hơn kỳ vọng.]

5 Challenge 4: Sắp xếp mảng một chiều

Yêu cầu bài toán

Cho N bản ghi (v, label) , chương trình cần sắp xếp ổn định theo v giảm dần, sau đó theo label tăng dần từ điển. Nếu cả hai khóa bằng nhau, thứ tự xuất hiện ban đầu phải được giữ nguyên. Đề bài không cho phép sử dụng các hàm sắp xếp dựng sẵn của Python.

5.1 Hiện thực đã xây dựng

Trong `challenge/challenge4.py`, mỗi bản ghi được biến đổi thành $(-v, \text{label})$. Cách biểu diễn này cho phép dùng thứ tự tăng dần tự nhiên của tuple để đạt được yêu cầu v giảm dần và label tăng dần. Thuật toán sắp xếp cục bộ là bottom-up merge sort do nhóm tự hiện thực.

Khi số lượng bản ghi đủ lớn, chương trình chia mảng thành các chunk liên tiếp. Mỗi worker sắp xếp một chunk bằng `bottom_up_merge_sort`. Sau đó, tiến trình chính lần lượt merge các chunk đã sắp xếp bằng hàm `merge_two` cho đến khi thu được danh sách cuối cùng.

5.2 Phân tích song song hóa

Merge sort cho phép tách giai đoạn sắp xếp cục bộ trên các đoạn độc lập. Đây là phần được song song hóa trong mã nguồn hiện tại. Giai đoạn merge phụ thuộc vào kết quả của các chunk, nên được thực hiện sau khi toàn bộ worker hoàn tất.

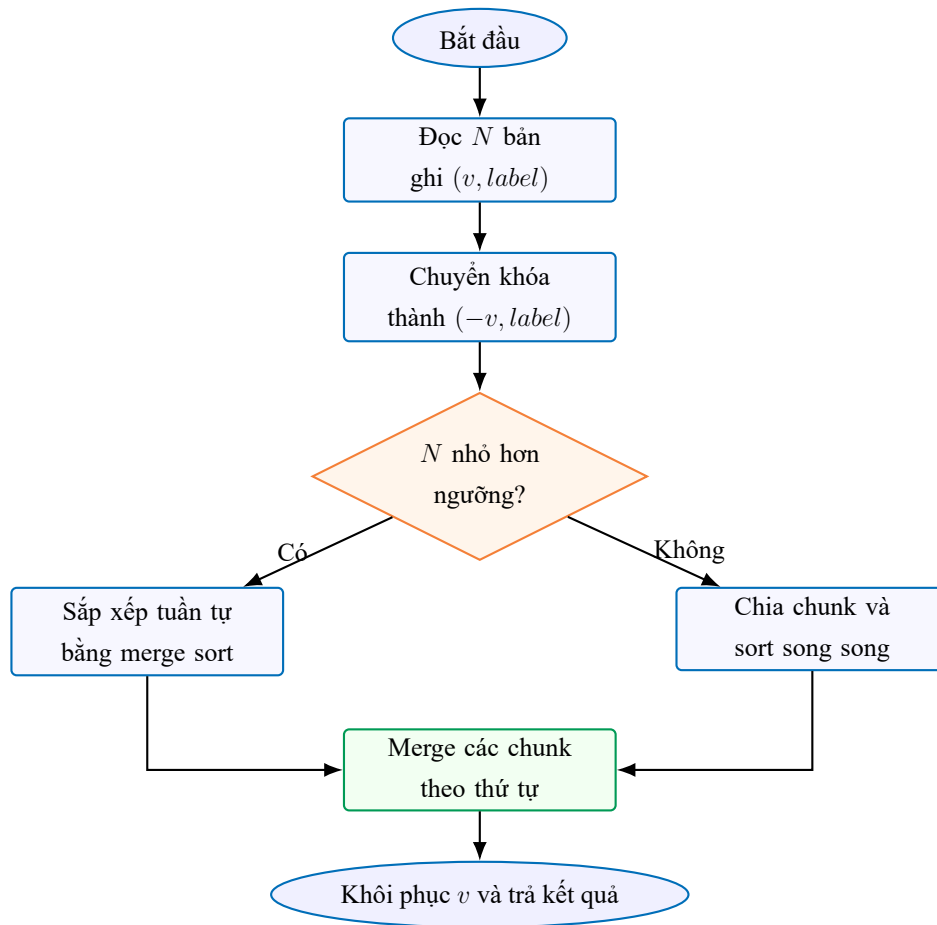
Thiết kế này ưu tiên tính đơn giản và độ tin cậy. Việc để tiến trình chính merge kết quả giúp giảm số lần truyền dữ liệu qua lại giữa các process. Đổi lại, khi N rất lớn, giai đoạn merge tuần tự có thể trở thành nút thắt hiệu năng.

5.3 Thiết kế thuật toán song song

Listing 4: Pseudocode song song cho Challenge 4

```
1 MAIN(path):  
2     records = read_records(path)  
3     keys = [(-value, label) for each record]  
4     sorted_keys = parallel_sort(keys)  
5     return restore_original_value(sorted_keys)  
6  
7 parallel_sort(records):  
8     if len(records) < threshold:  
9         return bottom_up_merge_sort(records)  
10  
11     ranges = split_array_ranges(records)  
12     chunks = pool.map(sort_range, ranges)  
13     return merge_chunks(chunks)
```

5.4 Lưu đồ thuật toán



Hình 4: Luồng xử lý song song của Challenge 4

5.5 Giải thích vai trò các bước

Challenge 4 gồm hai pha chính: sắp xếp cục bộ và merge toàn cục. Trước khi chia việc, mỗi bản ghi $(v, label)$ được đổi thành khóa $(-v, label)$. Phép đổi dấu v biến yêu cầu sắp xếp giảm dần theo v thành sắp xếp tăng dần theo khóa thứ nhất; khóa `label` vẫn giữ thứ tự tăng dần từ điển.

Trong pha song song, tiến trình chính chia mảng thành các chunk liên tiếp. Mỗi worker chạy merge sort tự hiện thực trên một chunk và trả về chunk đã sắp xếp. Các worker không tương tác trực tiếp với nhau vì thứ tự bên trong mỗi chunk chỉ phụ thuộc vào dữ liệu của chunk đó. Sau đó, tiến trình chính merge các chunk theo thứ tự. Đây là bước quyết định tính đúng của toàn bộ danh sách, vì các chunk riêng lẻ đã có thứ tự nhưng chưa tạo thành một thứ tự toàn cục.

Vai trò của bước merge đặc biệt quan trọng đối với tính ổn định. Khi hai bản ghi có khóa bằng nhau, phần tử đến từ nửa trái được chọn trước. Do các chunk ban đầu giữ thứ tự xuất hiện trong input và được merge theo thứ tự từ trái sang phải, các bản ghi trùng cả v và `label` vẫn giữ thứ tự tương đối ban đầu.

5.6 Tiềm năng tăng tốc và ví dụ minh họa

Sắp xếp có chi phí $\mathcal{O}(N \log N)$. Khi N lớn, việc sắp xếp cục bộ các chunk song song có thể giảm đáng kể thời gian của pha sort. Tuy nhiên, speedup bị giới hạn bởi pha merge ở tiến trình chính và chi phí truyền các chunk đã sắp xếp từ worker về. Vì vậy, tiềm năng tăng tốc tốt nhất thường xuất hiện ở input lớn, còn input nhỏ có thể không có lợi do overhead process.

Ví dụ với các bản ghi $(5, QA)$, $(5, QB)$, $(-1, ID)$, $(9, QA)$, sau khi đổi khóa ta có $(-5, QA)$, $(-5, QB)$, $(1, ID)$, $(-9, QA)$. Nếu chia thành hai chunk, worker 1 sắp $(5, QA)$, $(5, QB)$ và worker 2 sắp $(-1, ID)$, $(9, QA)$ thành $(9, QA)$, $(-1, ID)$. Bước merge toàn cục tạo kết quả $(9, QA)$, $(5, QA)$, $(5, QB)$, $(-1, ID)$. Cặp $(5, QA)$ và $(5, QB)$ minh họa tiêu chí phụ theo label; nếu hai bản ghi hoàn toàn trùng khóa, thứ tự ban đầu được giữ nhờ merge ổn định.

5.7 Tính ổn định

Tính ổn định được bảo đảm trong bước merge. Khi hai phần tử có khóa bằng nhau, hàm merge chọn phần tử ở nửa trái trước nhờ điều kiện so sánh $a \leq b$. Vì các chunk được tạo theo thứ tự xuất hiện ban đầu và được merge theo đúng thứ tự đó, các bản ghi trùng khóa vẫn giữ được thứ tự tương đối ban đầu.

5.8 Dữ liệu, phụ thuộc và chi phí

- **Phân phối dữ liệu:** mảng bản ghi được chia thành các đoạn liên tiếp; worker xử lý một đoạn.
- **Phụ thuộc dữ liệu:** các chunk độc lập trong giai đoạn sort, nhưng phụ thuộc khi merge.
- **Cân bằng tải:** chia theo số lượng bản ghi giúp tải giữa các worker tương đối đồng đều.
- **Chi phí giao tiếp:** đáng kể vì mỗi worker trả về một danh sách đã sắp xếp; merge cuối có độ phức tạp $\mathcal{O}(N \log p)$.

5.9 Đánh giá hiệu năng

Bảng 6: Kết quả đo hiệu năng cho Challenge 4

N	Số worker	T_{seq} (s)	T_{par} (s)	Speedup	Efficiency
10^4	[CẦN ĐIỀN: p]	[CẦN ĐIỀN: tseq]	[CẦN ĐIỀN: tpar]	[CẦN ĐIỀN: speedup]	[CẦN ĐIỀN: eff]
10^5	[CẦN ĐIỀN: p]	[CẦN ĐIỀN: tseq]	[CẦN ĐIỀN: tpar]	[CẦN ĐIỀN: speedup]	[CẦN ĐIỀN: eff]
10^6	[CẦN ĐIỀN: p]	[CẦN ĐIỀN: tseq]	[CẦN ĐIỀN: tpar]	[CẦN ĐIỀN: speedup]	[CẦN ĐIỀN: eff]

Nhận xét. **[CẦN ĐIỀN: Nêu rõ ngưỡng N nhỏ có thể chạy tuần tự nhanh hơn do overhead. Với N lớn, sort chunk song song có lợi, nhưng merge tuần tự và chi phí truyền danh sách khiến speedup không tuyến tính.]**

6 Challenge 5: Tìm kiếm trong mảng số nguyên lớn

Yêu cầu bài toán

Input gồm khóa cần tìm K và một mảng số nguyên A có thể chứa tối đa 10^6 phần tử. Chương trình cần trả về chỉ số nhỏ nhất i sao cho $A[i] = K$; nếu không tồn tại, kết quả là -1 . Đề bài yêu cầu tự chia mảng và không sử dụng hàm tìm kiếm dựng sẵn.

6.1 Hiện thực đã xây dựng

Trong `challenge/challenge5.py`, chương trình đọc toàn bộ input ở dạng byte. Token đầu tiên được xem là khóa cần tìm K ; phần dữ liệu còn lại biểu diễn mảng A dưới dạng các token số nguyên phân tách bởi khoảng trắng hoặc xuống dòng. Thay vì chuyển toàn bộ mảng thành danh sách số nguyên ngay từ đầu, mã nguồn tìm trực tiếp vị trí xuất hiện của token khóa trên buffer byte, sau đó quy đổi vị trí byte này về chỉ số phần tử trong mảng.

Hàm `parallel_find` xử lý riêng phần tử đầu tiên của mảng để bắt trường hợp K xuất hiện ngay tại đầu vùng dữ liệu. Nếu chưa tìm thấy, vùng dữ liệu còn lại được chia thành nhiều khoảng byte liên tiếp bằng hàm `make_ranges`. Mỗi worker chạy `worker_find`, tìm các vị trí có dạng token độc lập, tức là khóa phải đứng sau dấu cách hoặc ký tự xuống dòng và phía sau khóa phải là biên token hợp lệ. Kết quả worker trả về là offset byte nhỏ nhất trong khoảng được giao, hoặc -1 nếu không tìm thấy.

6.2 Phân tích song song hóa

Đơn vị công việc song song là các khoảng byte liên tiếp trong vùng dữ liệu của mảng. Các worker chỉ đọc buffer dùng chung và không ghi vào dữ liệu đầu vào, do đó không phát sinh xung đột ghi. Sau khi các worker hoàn tất, tiến trình chính chọn offset nhỏ nhất trong các kết quả hợp lệ. Offset này tương ứng với lần xuất hiện đầu tiên của khóa trong toàn bộ mảng.

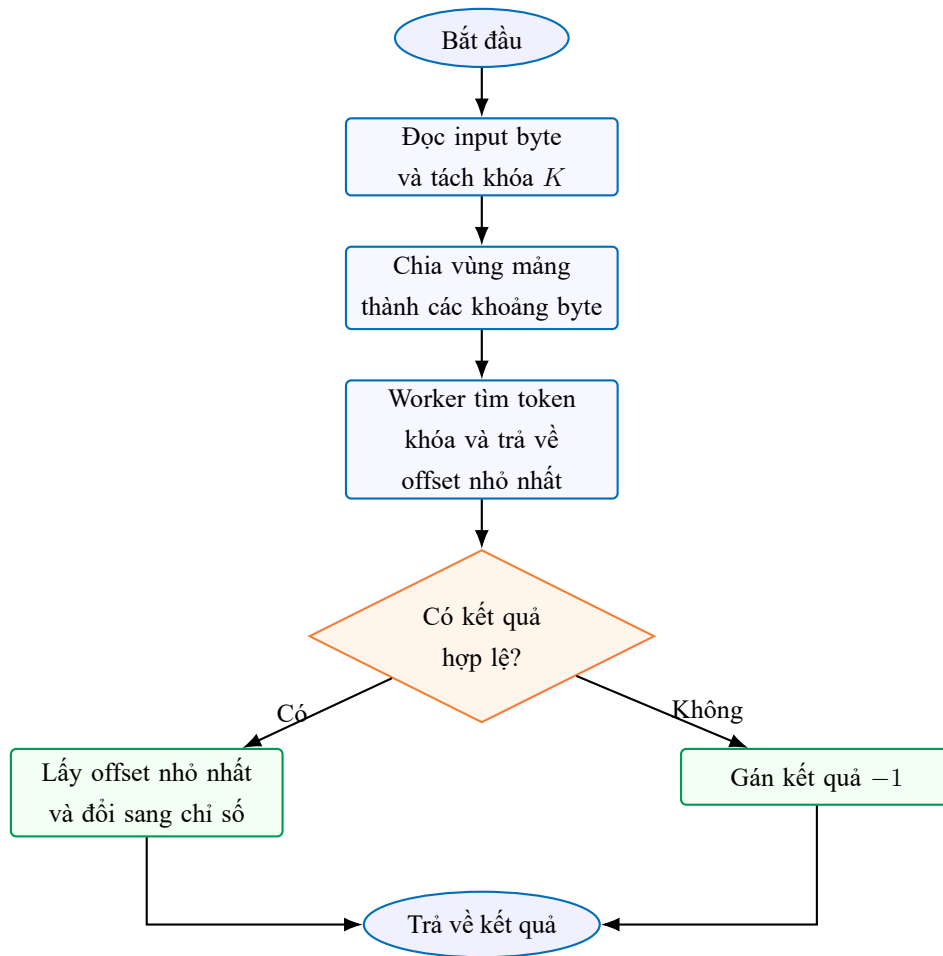
Do việc chia theo byte có thể cắt qua giữa một token, mỗi worker không chỉ kiểm tra chuỗi khóa mà còn kiểm tra biên token trước và sau khóa. Cách kiểm tra này ngăn trường hợp khóa 20 bị nhận nhầm bên trong token 120 hoặc 200. Sau bước reduce, hàm `index_from_offset` đếm số token đứng trước offset tìm được để trả về chỉ số mảng theo quy ước bắt đầu từ 0.

6.3 Thiết kế thuật toán song song

Listing 5: Pseudocode song song cho Challenge 5

```
1 MAIN(path):
2     data = read_all_bytes(path)
3     key = first_token(data)
4     arr_start = first_token_after_key(data)
5
6     pos = parallel_find(data, arr_start, key)
7     if pos == -1:
8         return -1
9     return index_from_offset(data, arr_start, pos)
10
11 parallel_find(data, arr_start, key):
12     if first_array_token_equals_key(data, arr_start, key):
13         return arr_start
14
15     ranges = split_byte_ranges(arr_start + 1, len(data))
16     offsets = pool.map(worker_find, ranges)
17     return minimum_valid_offset(offsets)
18
19 worker_find(lo, hi):
20     for each occurrence of key pattern in data[lo:hi]:
21         if occurrence is a complete token:
22             return occurrence_offset
23     return -1
```

6.4 Lưu đồ thuật toán



Hình 5: Luồng xử lý song song của Challenge 5

6.5 Giải thích vai trò các bước

Tiến trình chính thực hiện ba bước chuẩn bị: đọc buffer byte, tách token khóa K và xác định vị trí bắt đầu của mảng. Sau đó, chương trình kiểm tra riêng token đầu tiên trong mảng. Bước này cần thiết vì các worker tìm khóa thông qua mẫu có ký tự phân tách đứng trước, trong khi phần tử đầu tiên của mảng không nhất thiết có ký tự phân tách trước nó trong vùng tìm kiếm.

Khi cần chạy song song, tiến trình chính chia vùng mảng thành các khoảng byte và gán cho worker. Mỗi worker tìm các mẫu `space + key` hoặc `newline + key`, rồi kiểm tra biên bên phải để chắc chắn khóa là một token hoàn chỉnh. Worker chỉ trả về offset byte nhỏ nhất trong khoảng của mình. Tiến trình chính chọn offset nhỏ nhất trên toàn bộ kết quả, vì offset nhỏ hơn tương ứng với token xuất hiện sớm hơn trong input. Cuối cùng, `index_from_offset` đếm số token đứng trước offset này để đổi từ vị trí byte sang chỉ số mảng.

6.6 Tiềm năng tăng tốc và ví dụ minh họa

Tìm kiếm tuyến tính có độ phức tạp $\mathcal{O}(N)$. Phiên bản song song có thể giảm thời gian quét khi mảng rất lớn và khóa xuất hiện muộn hoặc không xuất hiện, vì nhiều đoạn dữ liệu được kiểm tra đồng thời. Nếu khóa nằm ngay đầu mảng, phiên bản hiện tại trả về sớm trước khi tạo pool, nhờ đó tránh overhead không cần thiết. Nếu khóa nằm gần giữa hoặc cuối mảng, speedup phụ thuộc vào kích thước input, số worker và chi phí quét byte.

Ví dụ với input có khóa $K = 20$ và mảng `10 25 5 30 15 40 20 5 35 45 0`. Sau khi tách khóa, vùng mảng được chia cho các worker. Worker nào phụ trách đoạn chứa token `20` sẽ trả về offset của token đó. Các worker khác trả về -1 . Tiến trình chính chọn offset hợp lệ duy nhất, sau đó đếm các token đứng trước nó: `10, 25, 5, 30, 15, 40`. Vì có 6 token trước `20`, kết quả trả về là chỉ số 6.

6.7 Dữ liệu, phụ thuộc và chi phí

- **Phân phối dữ liệu:** buffer input được lưu trong biến toàn cục; worker chỉ nhận cặp chỉ số byte (`lo`, `hi`).
- **Phụ thuộc dữ liệu:** các khoảng tìm kiếm độc lập; bước reduce chỉ chọn offset nhỏ nhất trong các kết quả hợp lệ.
- **Cân bằng tải:** chia theo số byte giúp kích thước mỗi phần việc gần bằng nhau, nhưng thời gian thực tế còn phụ thuộc vào mật độ xuất hiện của khóa và độ dài token.
- **Chi phí giao tiếp:** thấp, vì mỗi worker chỉ trả về một offset byte hoặc -1 .

6.8 Đánh giá hiệu năng

Bảng 7: Kết quả đo hiệu năng cho Challenge 5

Kích thước	Số worker	T_{seq} (s)	T_{par} (s)	Speedup	Efficiency
[CẦN ĐIỀN: size]	[CẦN ĐIỀN: p]	[CẦN ĐIỀN: tseq]	[CẦN ĐIỀN: tpar]	[CẦN ĐIỀN: speedup]	[CẦN ĐIỀN: eff]
[CẦN ĐIỀN: size]	[CẦN ĐIỀN: p]	[CẦN ĐIỀN: tseq]	[CẦN ĐIỀN: tpar]	[CẦN ĐIỀN: speedup]	[CẦN ĐIỀN: eff]
[CẦN ĐIỀN: size]	[CẦN ĐIỀN: p]	[CẦN ĐIỀN: tseq]	[CẦN ĐIỀN: tpar]	[CẦN ĐIỀN: speedup]	[CẦN ĐIỀN: eff]

Nhận xét. [CẦN ĐIỀN: Điền số liệu cho các trường hợp K xuất hiện đầu mảng, giữa mảng, cuối mảng và không xuất hiện. Cần giải thích vì sao việc chọn offset nhỏ nhất bảo đảm trả về chỉ số đầu tiên, đồng thời phân tích chi phí quy đổi offset về chỉ số mảng.]

7 Challenge 6: Phân tích không gian nhúng

Yêu cầu bài toán

Với Q bộ dữ liệu, mỗi bộ gồm nhiều điểm 2D, chương trình cần tìm khoảng cách Euclid nhỏ nhất giữa hai điểm bất kỳ trong từng bộ. Kết quả trả về là danh sách Q số thực, mỗi số được làm tròn đến 4 chữ số thập phân.

7.1 Hiện thực đã xây dựng

Trong `challenge/challenge6.py`, toàn bộ input được đọc thành danh sách số nguyên. Chương trình xác định vùng dữ liệu của từng test case và lưu lại trong `_G_RANGES`. Mỗi điểm (x, y) được mã hóa thành một số nguyên 64-bit sau khi cộng offset, giúp kiểm tra trùng điểm nhanh bằng `set`.

Với từng bộ dữ liệu không có điểm trùng, hàm `_closest_pair_core` sử dụng kỹ thuật hash grid. Một tập điểm đầu được chọn làm mẫu để ước lượng khoảng cách tốt nhất ban đầu. Sau đó, mỗi điểm mới chỉ cần so sánh với các điểm nằm trong ô lưới hiện tại và các ô lân cận. Khi tìm được khoảng cách nhỏ hơn, kích thước ô lưới được cập nhật để giảm số phép so sánh tiếp theo.

7.2 Phân tích song song hóa

Các bộ dữ liệu trong input độc lập với nhau. Vì vậy, đơn vị song song phù hợp nhất là từng test case. Mỗi worker nhận chỉ số của một bộ dữ liệu, tự tạo tập điểm, xử lý closest pair và trả về kết quả kèm chỉ số case. Tiến trình chính dùng chỉ số này để đặt kết quả vào đúng vị trí trong danh sách output.

Để cải thiện cân bằng tải, các case được sắp theo kích thước giảm dần trước khi đưa vào pool. Cách làm này giúp các case lớn được xử lý sớm, hạn chế tình trạng một worker phải xử lý case lớn ở cuối trong khi các worker khác đã hoàn tất.

7.3 Thiết kế thuật toán song song

Listing 6: Pseudocode song song cho Challenge 6

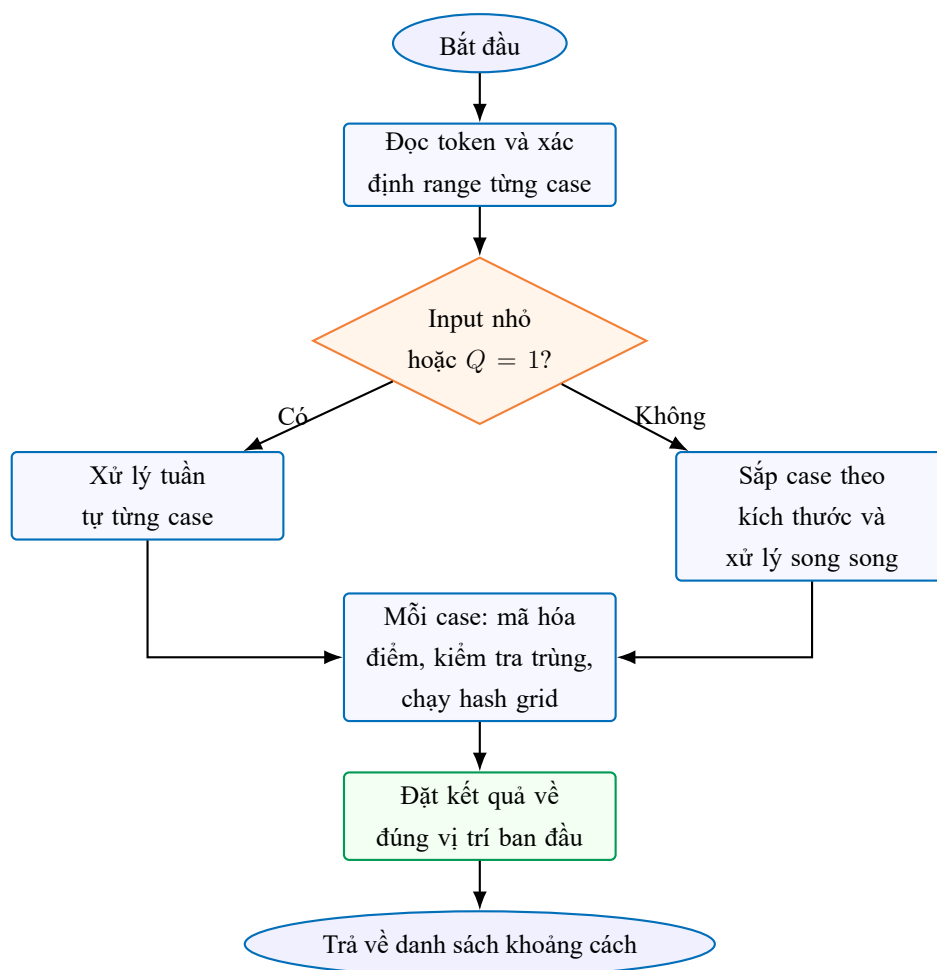
```
1 MAIN(path):  
2     tokens = parse_all_ints(path)  
3     ranges = locate_each_case(tokens)  
4
```

```

5     if input_is_small:
6         return [solve_case(i) for i in range(Q)]
7
8     order = cases_sorted_by_size_descending()
9     for case_idx, ans in pool.imap_unordered(solve_case, order):
10         answers[case_idx] = ans
11     return answers
12
13 solve_case(case_idx):
14     points = encode_points_as_set(case_idx)
15     if duplicate_exists:
16         return 0.0
17     return closest_pair_hash_grid(points)

```

7.4 Lưu đồ thuật toán



Hình 6: Luồng xử lý song song của Challenge 6

7.5 Giải thích vai trò các bước

Tiến trình chính đọc toàn bộ input, xác định vị trí dữ liệu của từng test case và lưu các khoảng này để worker có thể truy cập đúng phần dữ liệu cần xử lý. Mỗi test case độc lập về dữ liệu và kết quả, nên worker chỉ cần nhận chỉ số case, tạo tập điểm cục bộ và trả về khoảng cách nhỏ nhất của case đó.

Trong từng worker, bước mã hóa điểm giúp biến cặp tọa độ (x, y) thành một khóa nguyên duy nhất để kiểm tra trùng điểm nhanh. Nếu phát hiện hai điểm trùng nhau, khoảng cách nhỏ nhất bằng 0.0 và worker có thể trả về ngay. Nếu không có điểm trùng, worker sử dụng hash grid: chia mặt phẳng thành các ô theo kích thước lưới hiện tại, mỗi điểm mới chỉ so sánh với các điểm trong ô của nó và các ô lân cận. Khi tìm được khoảng cách tốt hơn, kích thước lưới được cập nhật để giảm số ứng viên cần so sánh về sau.

Tương tác giữa các tiến trình diễn ra ở mức test case. Worker trả về cặp $(case_idx, ans)$; tiến trình chính dùng $case_idx$ để đặt kết quả vào đúng vị trí ban đầu. Cách này cho phép dùng `imap_unordered`: worker nào xong trước có thể trả kết quả trước mà không làm sai thứ tự output.

7.6 Tiềm năng tăng tốc và ví dụ minh họa

Tiềm năng tăng tốc phụ thuộc mạnh vào số lượng test case Q và phân bố kích thước từng case. Khi có nhiều case độc lập, tiến trình chính có thể giao các case khác nhau cho nhiều worker, nhờ đó thời gian tổng gần với thời gian xử lý của nhóm case nặng nhất thay vì tổng thời gian của tất cả case. Việc sắp case lớn trước giúp giảm nguy cơ một case lớn còn lại ở cuối làm toàn bộ chương trình phải chờ.

Ví dụ với $Q = 2$, case 1 gồm các điểm $(4, 0), (8, 9), (0, 4), (2, 0)$ và case 2 gồm $(2, 0), (1, 6), (3, 9), (3, 10)$. Hai worker có thể xử lý hai case này độc lập. Worker của case 1 tìm được khoảng cách nhỏ nhất 2.0 giữa $(4, 0)$ và $(2, 0)$; worker của case 2 tìm được khoảng cách nhỏ nhất 1.0 giữa $(3, 9)$ và $(3, 10)$. Tiến trình chính đặt hai kết quả theo đúng chỉ số case và trả về $[2.0, 1.0]$.

7.7 Dữ liệu, phụ thuộc và chi phí

- **Phân phối dữ liệu:** danh sách token và range từng case là dữ liệu đọc chung; worker nhận chỉ số case cần xử lý.
- **Phụ thuộc dữ liệu:** các case độc lập; trong một case, hash grid là cấu trúc cục bộ của worker.
- **Cân bằng tải:** sắp xếp case lớn trước giúp giảm thời gian chờ ở cuối quá trình xử lý.
- **Chi phí giao tiếp:** thấp vì mỗi worker chỉ trả về một số thực và chỉ số case.

- **Độ đúng:** cần bảo đảm khi cập nhật kích thước lưới, các điểm đã duyệt được đưa lại vào lưới theo kích thước mới.

7.8 **Đánh giá hiệu năng**

Bảng 8: Kết quả đo hiệu năng cho Challenge 6

Kích thước	Số worker	T_{seq} (s)	T_{par} (s)	Speedup	Efficiency
[CẦN ĐIỀN: size]	[CẦN ĐIỀN: p]	[CẦN ĐIỀN: tseq]	[CẦN ĐIỀN: tpar]	[CẦN ĐIỀN: speedup]	[CẦN ĐIỀN: eff]
[CẦN ĐIỀN: size]	[CẦN ĐIỀN: p]	[CẦN ĐIỀN: tseq]	[CẦN ĐIỀN: tpar]	[CẦN ĐIỀN: speedup]	[CẦN ĐIỀN: eff]
[CẦN ĐIỀN: size]	[CẦN ĐIỀN: p]	[CẦN ĐIỀN: tseq]	[CẦN ĐIỀN: tpar]	[CẦN ĐIỀN: speedup]	[CẦN ĐIỀN: eff]

Nhận xét. [CẦN ĐIỀN: Điền số liệu theo số case Q, tổng số điểm và phân bố kích thước case. Cần giải thích vì sao Q nhỏ khó tận dụng song song theo test case.]

8 **Kết luận**

Qua sáu challenge, có thể thấy hiệu quả của xử lý song song phụ thuộc trực tiếp vào mức độ độc lập của các đơn vị công việc và chi phí phối hợp giữa các worker. Những bài toán có cấu trúc dữ liệu độc lập rõ ràng như lọc mảng, xử lý truy vấn Fibonacci hoặc tìm kiếm tuyến tính thường phù hợp với mô hình chia đoạn. Ngược lại, các bài toán có kết quả trung gian lớn như nhân ma trận hoặc sắp xếp cần cân nhắc kỹ chi phí truyền dữ liệu và bước gom kết quả.

Về mặt thiết kế, nhóm đã ưu tiên các chiến lược chia việc đơn giản, dễ kiểm chứng và phù hợp với ràng buộc của hệ thống chấm. Các worker chủ yếu xử lý trên những vùng dữ liệu không giao nhau, nhờ đó giảm nhu cầu khóa đồng bộ. Đối với các bài toán cần bảo toàn thứ tự kết quả, chương trình sử dụng offset hoặc chỉ số test case để đặt kết quả về đúng vị trí ban đầu.

8.1 **Tổng hợp kết quả**

Bảng 9: Tổng hợp speedup tốt nhất sau khi đo

Challenge	Chiến lược chính	Kích thước tốt nhất	Speedup tốt nhất	Nhận xét ngắn
1	Chia byte-range, reduce tổng	[CẦN ĐIỀN: size]	[CẦN ĐIỀN: speedup]	[CẦN ĐIỀN: nhận xét]
2	Chia truy vấn, ghi vào RawArray	[CẦN ĐIỀN: size]	[CẦN ĐIỀN: speedup]	[CẦN ĐIỀN: nhận xét]
3	Chia theo cụm dòng ma trận	[CẦN ĐIỀN: size]	[CẦN ĐIỀN: speedup]	[CẦN ĐIỀN: nhận xét]
4	Sort chunk song song, merge kết quả	[CẦN ĐIỀN: size]	[CẦN ĐIỀN: speedup]	[CẦN ĐIỀN: nhận xét]
5	Chia đoạn tìm kiếm	[CẦN ĐIỀN: size]	[CẦN ĐIỀN: speedup]	[CẦN ĐIỀN: nhận xét]
6	Chia theo bộ dữ liệu điểm	[CẦN ĐIỀN: size]	[CẦN ĐIỀN: speedup]	[CẦN ĐIỀN: nhận xét]

8.2 Nhận định chung

Speedup kỳ vọng không tăng tuyến tính theo số worker trong mọi trường hợp. Với dữ liệu nhỏ, chi phí tạo tiến trình và truyền dữ liệu có thể lớn hơn lợi ích song song hóa. Với dữ liệu lớn, hiệu quả cải thiện rõ hơn, nhưng vẫn bị giới hạn bởi băng thông bộ nhớ, chi phí pickle/IPC, bước reduce và mức cân bằng tải giữa các worker. Do đó, việc chọn ngưỡng chuyển từ tuần tự sang song song là một phần quan trọng trong thiết kế.

8.3 Checklist trước khi nộp

- Điền mã nhóm, key và thông tin thành viên ở `main.tex`.
- Cập nhật nội dung Challenge 5 nếu nhóm bổ sung mã nguồn cuối cùng cho challenge này.
- Đo và điền đầy đủ bảng thời gian tuần tự/song song cho từng challenge.
- Giải thích các trường hợp speedup thấp hoặc chương trình song song chậm hơn chương trình tuần tự.
- Build báo cáo thành PDF bằng XeLaTeX và kiểm tra lại mục lục, bảng, hình, số trang.
- Bảo đảm phiên bản tuần tự chỉ dùng để benchmark trong báo cáo, không nộp lên hệ thống chấm.

A Phụ lục: Mẫu đo hiệu năng

Phần này trình bày một mẫu đo thời gian tham khảo. Khi hoàn thiện báo cáo, nhóm cần thay thế hoặc đối chiếu với script đo thực tế đã sử dụng để bảo đảm số liệu có thể kiểm chứng.

Listing 7: Mẫu đo thời gian một hàm MAIN

```
1 from time import perf_counter
2
3 def measure(func, path, repeat=3):
4     times = []
5     last_result = None
6     for _ in range(repeat):
7         start = perf_counter()
8         last_result = func(path)
9         end = perf_counter()
10        times.append(end - start)
11    return sum(times) / len(times), last_result
```

B Phụ lục: Mẫu phiên bản tuần tự tham chiếu

Phiên bản tuần tự chỉ dùng để đo so sánh trong báo cáo, không nộp lên hệ thống chấm. Khi trình bày trong báo cáo, nhóm cần mô tả ngắn gọn phiên bản tuần tự tương ứng thay vì đưa toàn bộ mã nguồn nếu phần hiện thực quá dài.

Bảng 10: Phiên bản tuần tự tham chiếu đề xuất

Challenge	Thuật toán tuần tự	Ghi chú
1	Quét token một lượt, cộng trực tiếp	Cùng điều kiện lọc với bản song song.
2	Fast doubling cho từng truy vấn	Có thể dùng cùng hàm <code>_fib_pair</code> .
3	Ba vòng lặp nhân ma trận	Có thể vẫn dùng B^T để so sánh công bằng về truy cập bộ nhớ.
4	Bottom-up merge sort một luồng	Không dùng <code>sorted</code> hoặc <code>list.sort</code> .
5	Quét tuyến tính từ trái sang phải	Trả về ngay khi gặp K .
6	Thuật toán cùng hash grid chạy tuần tự từng case	Hoặc chia để trị tuần tự nếu nhóm hiện thực.

C Phụ lục: Checklist nội dung theo đề

Bảng 11: Đối chiếu yêu cầu đề bài với báo cáo

Yêu cầu	Vị trí trong báo cáo	Trạng thái
Phân tích phần song song hóa	Mỗi section challenge, tiểu mục “Phân tích song song hóa”	Đáp ứng
Phương pháp chia việc	Mỗi section challenge, tiểu mục phân tích và thuật toán	Đáp ứng
Phân phối và truy cập dữ liệu	Mỗi section challenge, tiểu mục “Dữ liệu, phụ thuộc và chi phí”	Đáp ứng
Phân tích phụ thuộc dữ liệu, cân bằng tải, chi phí giao tiếp	Mỗi section challenge	Đáp ứng
Thuật toán song song chi tiết	Pseudocode trong từng challenge	Đáp ứng
Sơ đồ/lưu đồ nếu cần	Lưu đồ TikZ trong từng section challenge	Đáp ứng
Giải thích vai trò từng bước và tương tác tiến trình	Tiểu mục “Giải thích vai trò các bước” trong từng challenge	Đáp ứng
Tiềm năng tăng tốc so với tuần tự	Tiểu mục “Tiềm năng tăng tốc và ví dụ minh họa” trong từng challenge	Đáp ứng
Ví dụ minh họa chi tiết	Tiểu mục “Tiềm năng tăng tốc và ví dụ minh họa” trong từng challenge	Đáp ứng
Đo thời gian chương trình song song	Bảng hiệu năng từng challenge	[CẦN ĐIỀN: Cần điền]
So sánh với chương trình tuần tự	Bảng hiệu năng từng challenge	[CẦN ĐIỀN: Cần điền]
Giải thích kết quả	Đoạn “Nhận xét” từng challenge	[CẦN ĐIỀN: Cần điền]
Mã nhóm, key, thành viên	Trang bìa	[CẦN ĐIỀN: Cần điền]